

LISTA 3a DE EXERCÍCIOS – Comunicação e Sincronização entre Processos – Exclusão mútua – Sleep e Wakeup

1. Em um sistema de multiprocessamento ou multiprogramação, como os processos podem se comunicar com outros processos?
2. Explique:
 - a. Condições de Corrida
 - b. Região crítica
 - c. Exclusão mútua
3. Explique cada uma das 4 condições necessárias para uma boa solução de exclusão mútua.
4. Explique porque desabilitar as interrupções da UCP, visando a exclusão mútua, não funciona em sistemas multiprocessadores.
5. Explique como funciona o mecanismo de variáveis de trava e porque ele é susceptível ao problema das condições de corrida.
6. Explique como funciona a instrução TSL (Test and Set Lock) e como ela pode ser usada na implementação da exclusão mútua.
7. Explique as desvantagens da implementação da exclusão mútua com espera ocupada com uso da instrução TSL.
8. Explique o funcionamento das primitivas Sleep() e Wakeup(), sem o uso do bit de Wakeup.
9. Escreva, em linguagem C, uma solução usando as primitivas Sleep() e Wakeup() (da questão anterior), para o problema do produtor/consumidor atuando sobre um buffer circular. Esta solução realmente funciona? Explique.
10. Qual a finalidade do bit de Wakeup?
11. As primitivas Sleep() e Wakeup(), implementadas com o uso do bit de Wakeup, conseguem prover uma solução para a exclusão mútua e a sincronização baseada em condições, considerando um número arbitrário de processos comunicantes? Explique.

Lista de Exercícios 3 - Comunicação e Sincronização entre Processos

Nome: Igor dos Reis Gomes

RA: 241025265

1- Os processos se comunicam entre si através de uma área de armazenamento comum, podendo ser na memória principal ou no disco e para acessar essa área o processo deve solicitar ao SO.

2-

a) Ocorrem quando múltiplos processos disputam o acesso a uma área de armazenamento compartilhado

b) É o trecho de código de um processo que envolve o acesso a dados ou variáveis compartilhadas

c) É a solução para o problema das condições de corrida. Consiste em permitir que mais de um processo acesse sua região crítica.

3- As condições são: se um processo sofrer preempção enquanto estiver em sua região crítica, nenhum outro processo pode entrar em sua própria região crítica. A exclusão mútua deve funcionar independente do hardware. Nenhum processo que está fora de sua região crítica pode bloquear outros processos de entrarem em suas próprias regiões críticas. E nenhum processo pode esperar para sempre para entrar em sua região crítica.

4- Porque desabilitando as interrupções de uma UCP não impede que outro processo, que está sendo executado em uma outra UCP, acesse a área de memória compartilhada.

5- Utiliza-se de uma variável compartilhada para indicar se um processo está em sua região crítica. Para um processo entrar na área compartilhada, é necessário testar essa flag e verificar se a área está disponível ou não. Essa técnica continua sendo problemática pois essa variável compartilhada fica suscetível a condições de corrida.

6- A instrução TSL é uma operação de hardware que lê o conteúdo da variável compartilhada, copia esse valor para um registrador e armazena um valor não nulo nessa variável. Ela pode ser usada na exclusão mútua pois elimina a condição de corrida presente na variável compartilhada de trava.

7- A principal desvantagem é o desperdício de tempo de UCP, já que um processo que está aguardando sua entrada na área compartilhada não fica bloqueado, mas sim executando um loop de verificações.

8- Sleep(): chamada de sistema que causa a suspensão de um processo corrente, colocando-o como bloqueado.

WakeUp(pid): chamada de sistema que acorda um processo, identificado pelo seu pid, que estava bloqueado, colocando-o em estado de pronto para execução.

9-

10-